	Nearest Neighbor: return the closest point (or optionally the closest `N` points, for any `N`) to a target point. The target point doesn't have to be in the container. Region: return all the points in the container within a certain region, either a hyper-sphere or hyper-rectangle.

8.4 Moving to k-dimensional Spaces

In section 8.2, we have showed that it is possible to efficiently solve the nearest neighbor problem in 1-D by using a binary search tree. If you read through the book and the appendices on core data structures, you should by now be familiar with binary trees. (If you did skip appendix C, this is your clue: go check out binary trees!).

When we go from 1-D to 2-D, however, the situation becomes slightly more complicated, since at each node we don't have a clear fork between two paths, namely left and right children: we have seen the same concept in ternary trees, where at each node, we have a fork with 3 paths (or more, in `n`-ary trees), and the direction we have to follow depends on the result of comparison with more than just `true`/`false` possible outcomes. But would an `n`-ary tree help us in this situation? Let's analyze what happens in 1-D and see if we can generalize.